Day 2 – Lab 2.3: Event Grid-Based Blob Trigger in Azure

Azure Functions · C# · .NET 10 Isolated Worker · Flex Consumption · Event Grid

Objective: Deploy the Blob → Queue → Blob workflow developed in Lab 2.2 to your Azure Function App and configure Event Grid to trigger the Blob-processing Function when a CSV file is uploaded.

Business scenario: Contoso Orders receives order files in Azure Blob Storage. Instead of continuously scanning the storage container, Event Grid detects new Blob creation events and notifies the Function App to start processing.

Resources created by you in the Day 2 setup guide:

Resource	Example
Resource Group	Existing shared Resource Group
Function App	`fn-user01-orders-d2`
Storage Account	`stuser01ordersd2`

Blob container	`incoming`
Blob container	`processed`
Queue	`orders`

Important: This lab uses the same Azure resources you created yourself. Do not create another Function App or Storage Account.

1. Understand the deployment change

Lab 2.2 used Azurite locally and the default Blob trigger polling mechanism.

For this Azure deployment, we will configure the Blob trigger to use Event Grid:

- Local Lab 2.2: polling-based Blob trigger.
- Azure Lab 2.3: Event Grid-based Blob trigger.
- Queue and Blob output logic: unchanged.

Flex Consumption supports the Event Grid-based Blob trigger source, not the polling-based source.

Microsoft Learn+1

2. Update the Blob trigger for Event Grid

Open `SplitCsv.cs`.

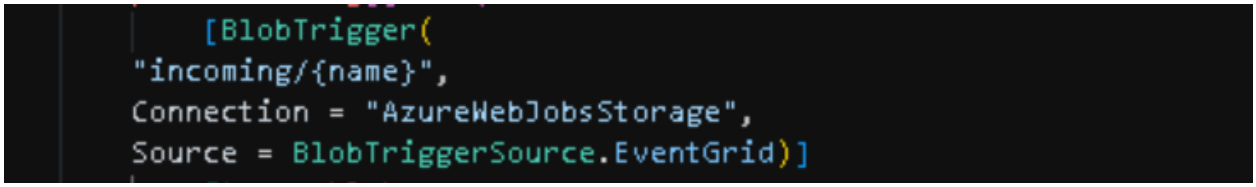
Find the existing Blob trigger attribute:

```
None
[BlobTrigger(
    "incoming/{name}",
    Connection = "AzureWebJobsStorage")]
```

Replace it with:

```
None
[BlobTrigger(
    "incoming/{name}",
    Connection = "AzureWebJobsStorage",
    Source = BlobTriggerSource.EventGrid)]
```

Keep the rest of `SplitCsv.cs` unchanged.



The complete attribute now tells Azure Functions to use Event Grid notifications instead of polling the container.

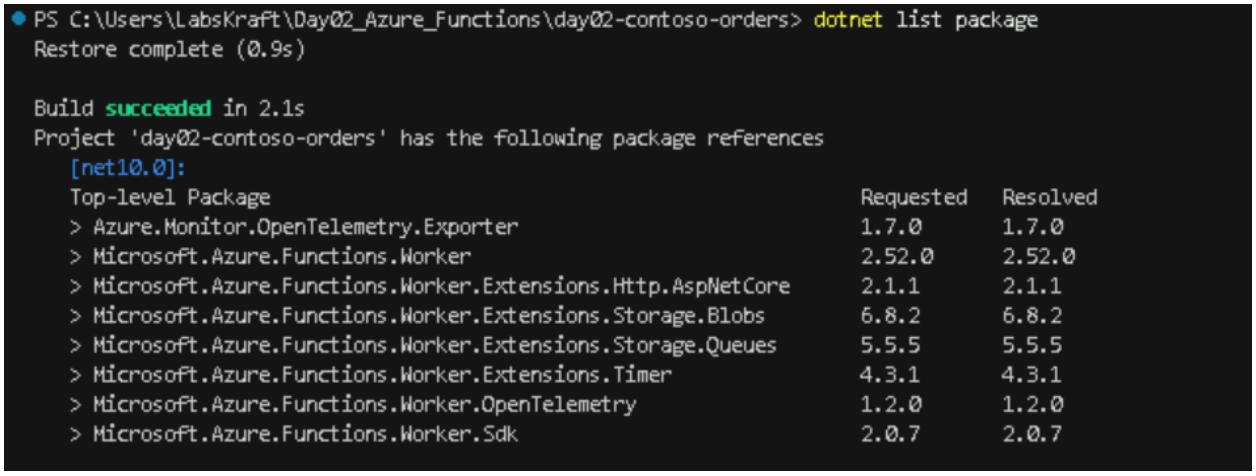
3. Verify the Storage extension

Run:

```
None
dotnet list package
```

Confirm that these packages are present:

```
None
Microsoft.Azure.Functions.Worker.Extensions.Storage.Blobs
Microsoft.Azure.Functions.Worker.Extensions.Storage.Queues
```

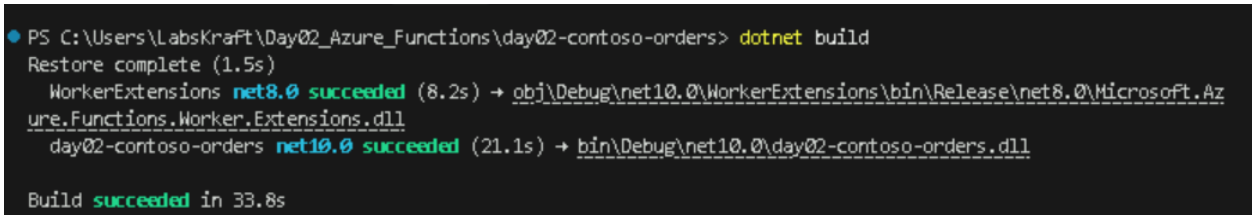


The Blob extension must be version 5.x or later for the Event Grid-based trigger.

Build the project:

```
None
dotnet build
```

Expected: `Build succeeded`.



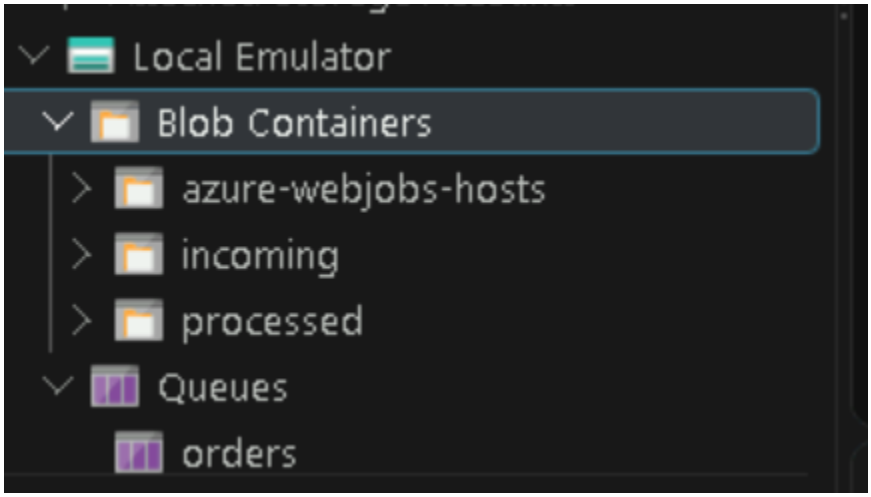
Microsoft reference: [Azure Blob Storage bindings for Azure Functions](#).

4. Verify your Azure Storage resources

Open Azure Portal and navigate to your Day 2 Storage Account.

Confirm these resources exist:

```
Storage Account
├── Blob Containers
│   ├── incoming
│   └── processed
└── Queues
    └── orders
```



If you haven't created them, create them now using the Storage Account's Containers and Queues sections.

5. Verify the Azure Function App storage configuration

Open your Day 2 Function App:

```
fn-user01-orders-d2
```

Navigate to:

Settings → Environment variables

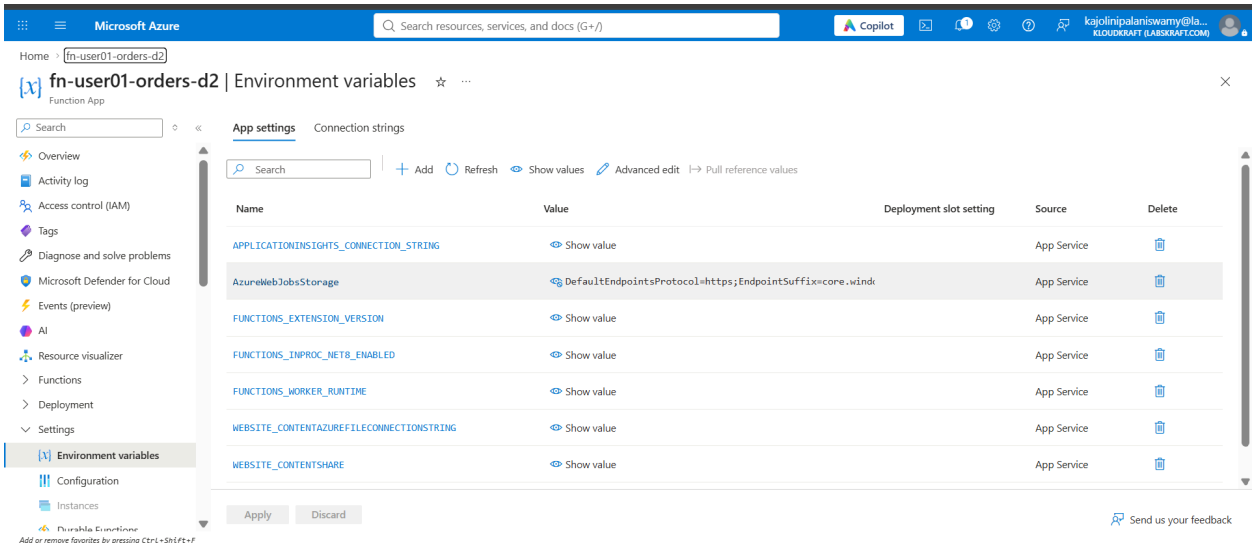
Verify that `AzureWebJobsStorage` points to your Azure Storage Account.

It must not contain:

```
UseDevelopmentStorage=true
```

That value is only for local Azurite execution.

Also ensure the deployed Function App has the permissions needed to read and write Blob and Queue data. If you use a connection string, it must belong to your Day 2 Storage Account.



Do not upload your local `local.settings.json` to Azure. It contains the local Azurite configuration.

6. Deploy the updated project

Stop the local Functions host using `Ctrl + C`.

From the project directory, execute:

```
None
func azure functionapp publish fn-user01-orders-d2
```

Replace the Function App name with your actual participant-specific name.

Wait for deployment to complete.

Open Azure Portal → Function App → Functions.

Confirm these Functions are listed:

- `StaleOrderSweep`
- `SplitCsv`
- `ProcessOrder`

```
PS C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders> func azure functionapp publish fn-user01-orde
rs-d2
Detected target framework: net10.0
  Determining projects to restore...
  All projects are up-to-date for restore.
  Determining projects to restore...
  Restored C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders\obj\release\net10.0\WorkerExtensions\W
orkerExtensions.csproj (in 1.8 sec).
  WorkerExtensions -> C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders\obj\release\net10.0\WorkerE
xtensions\bin\Release\net8.0\Microsoft.Azure.Functions.Worker.Extensions.dll
  day02-contoso-orders -> C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders\bin\publish\day02-conto
so-orders.dll

Build succeeded.

    0 Warning(s)
    0 Error(s)

Time Elapsed 00:00:18.53

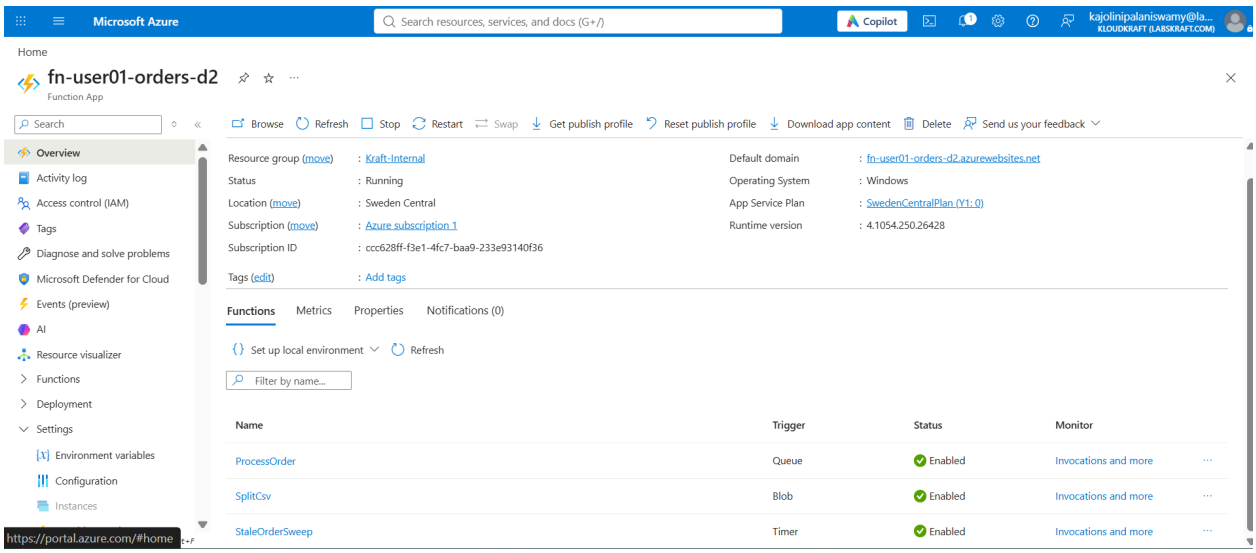
Getting site publishing info...
```

```
Getting site publishing info...
[2026-11-05T17:52:53.049Z] Starting the function app deployment...
Creating archive for current directory...
Uploading 5.2 MB [##############################################################################]
Upload completed successfully.
Deployment completed successfully.
[2026-11-05T17:54:24.690Z] Syncing triggers...
Functions in fn-user01-orders-d2:
    ProcessOrder - [queueTrigger]

    SplitCsv - [blobTrigger]

    StaleOrderSweep - [timerTrigger]

PS C:\Users\LabsKraft\Day02_Azure_Functions\day02-contoso-orders>
```



7. Retrieve the Blob extension system key

Event Grid needs an authenticated endpoint to deliver Blob events to your Function App.

1. Open your Function App in Azure Portal.
2. Navigate to Functions → App keys.
3. Find the System keys section.
4. Locate `blobs_extension`.
5. Copy its value securely.



Do not use the Function App master key.

The Blob extension system key protects the webhook endpoint.

Microsoft Learn+1

8. Construct the Event Grid webhook endpoint

Use this endpoint format:

```
None
https://fn-user01-orders-d2.azurewebsites.net/runtime/webhooks/blobs?functionName=Host.Functions.SplitCsv&code=<BLOB_EXTENSION_KEY>
```

Replace:

- `fn-user01-orders-d2` with your actual Function App name.

- `<BLOB_EXTENSION_KEY>` with the key copied in Step 7.

Keep the complete URL private because it contains an authorization key.

Important: The endpoint path is `/runtime/webhooks/blobs`, not `/runtime/webhooks/eventgrid`. This is a Blob trigger using Event Grid as its event source, not a standalone Event Grid-triggered Function.

Microsoft Learn+1

9. Create the Event Grid subscription

You will create the subscription yourself.

1. Open your Day 2 Storage Account.
2. Select Events.
3. Select + Event Subscription.
4. Configure:

Setting	Value
Name	`evsub-user01-orders-d2`
Event schema	Event Grid Schema
Event type	Blob Created

Endpoint type	Web Hook
Endpoint	Complete webhook URL from Step 8

5. Open the Filters tab.
6. Enable subject filtering.
7. Set Subject Begins With:

None

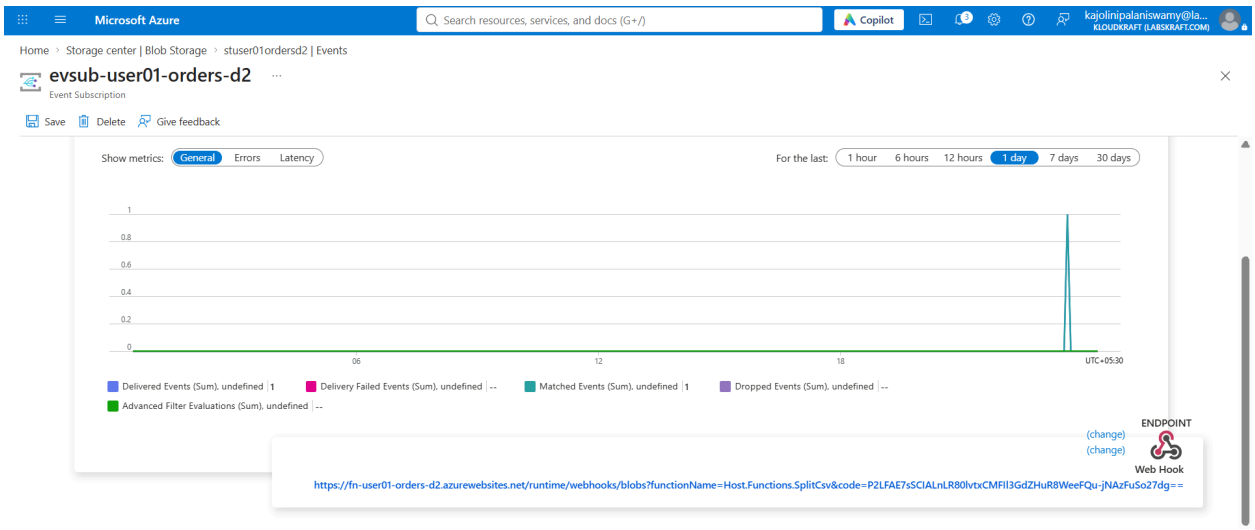
```
/blobServices/default/containers/incoming/blobs/
```

8. Set Subject Ends With:

None

```
.csv
```

9. Select Create.



This limits notifications to newly created CSV files inside your `incoming` container.

Microsoft's Blob-trigger tutorial uses the Blob extension webhook and Event Grid subscription with subject filters.

Microsoft Learn+1

If subscription creation fails because the Event Grid resource provider is not registered, check your subscription's Resource providers section and register `Microsoft.EventGrid` if your permissions allow it. You need permission to create the event subscription at the Storage Account scope.

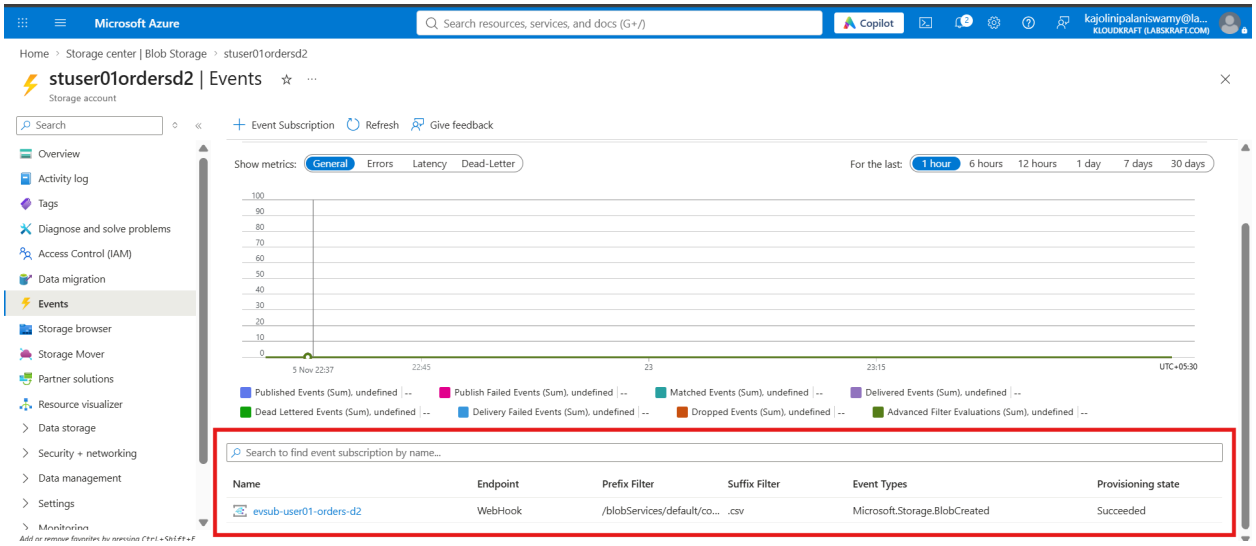
10. Validate the Event Grid subscription

Navigate to:

Storage Account → Events → Event Subscriptions

Confirm:

- `evsub-user01-orders-d2` exists.
- Provisioning state is `Succeeded`.

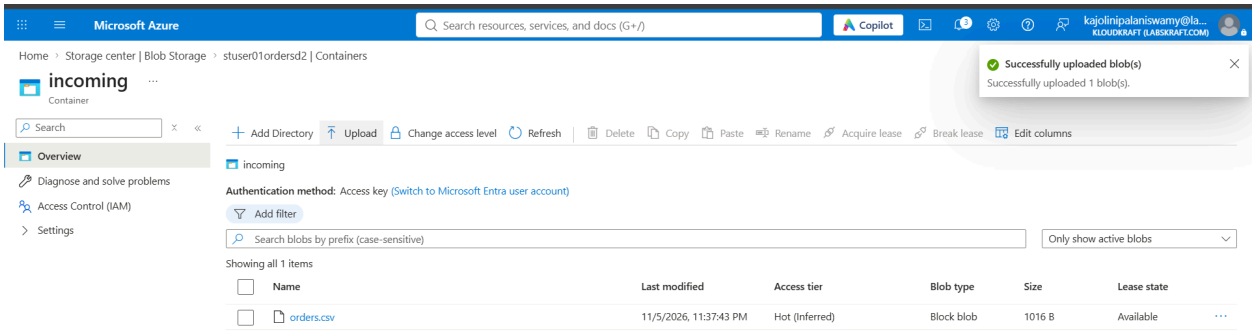


- Event type includes `Blob Created`.
- Endpoint points to your `SplitCsv` Function.
- Subject filters target `incoming/*.csv`.

11. Execute the end-to-end test

Use the existing `orders.csv` training asset containing 10 order records.

1. Open Storage Account → Containers → `incoming`.
2. Upload the CSV file.

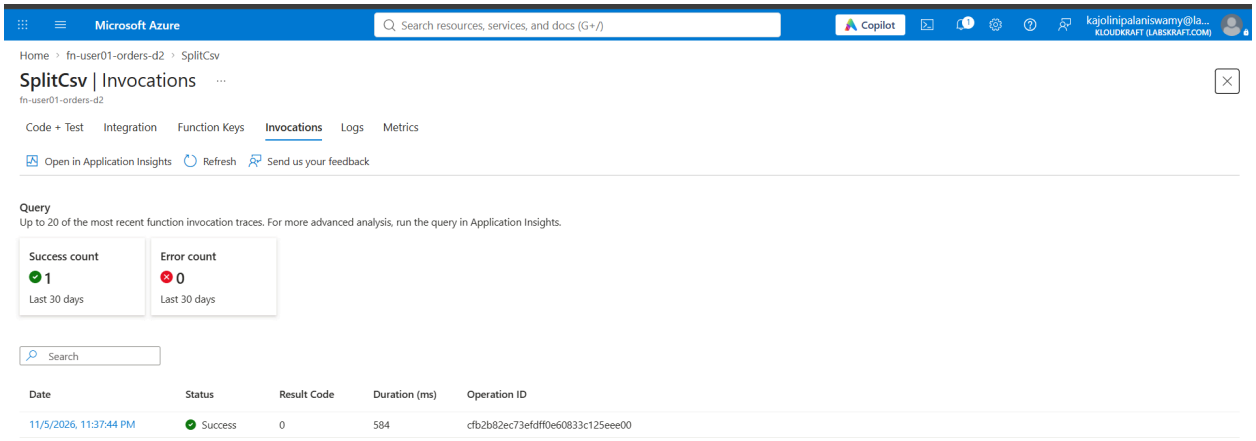


3. If `orders.csv` already exists, use a new filename such as:

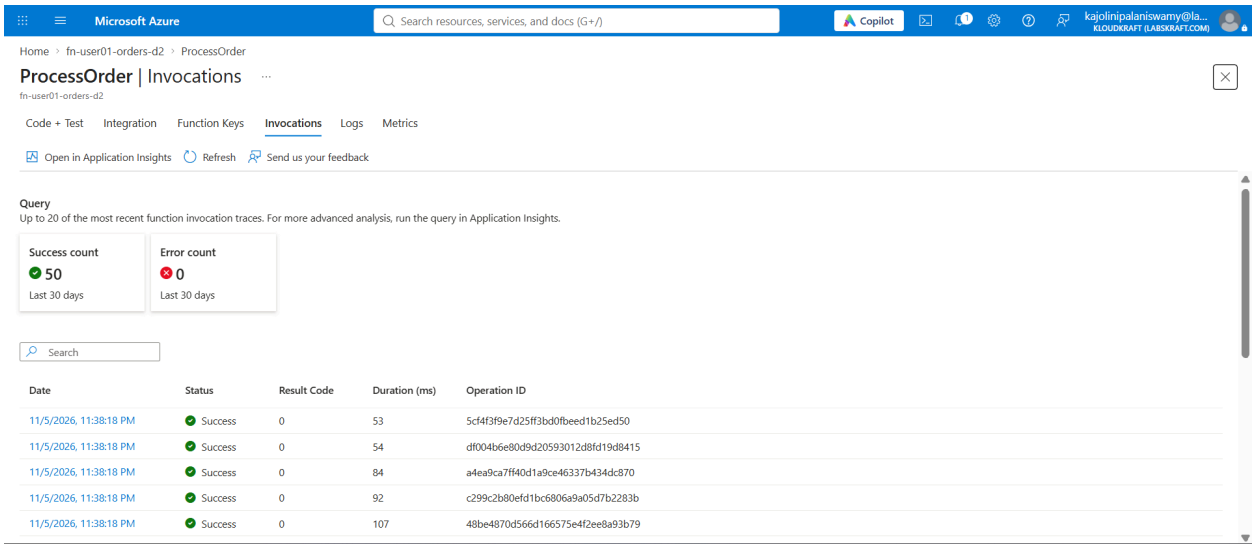
None

```
orders-user01-test01.csv
```

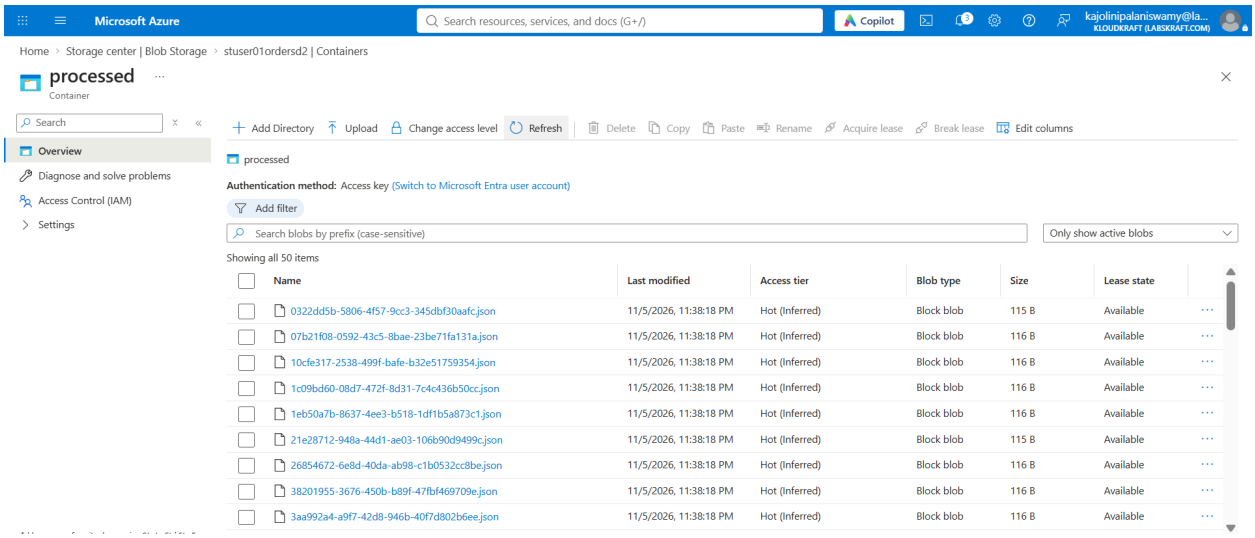
4. Wait for Event Grid to deliver the Blob Created event.
5. Open Function App → `SplitCsv` → Monitor.
6. Verify the invocation.



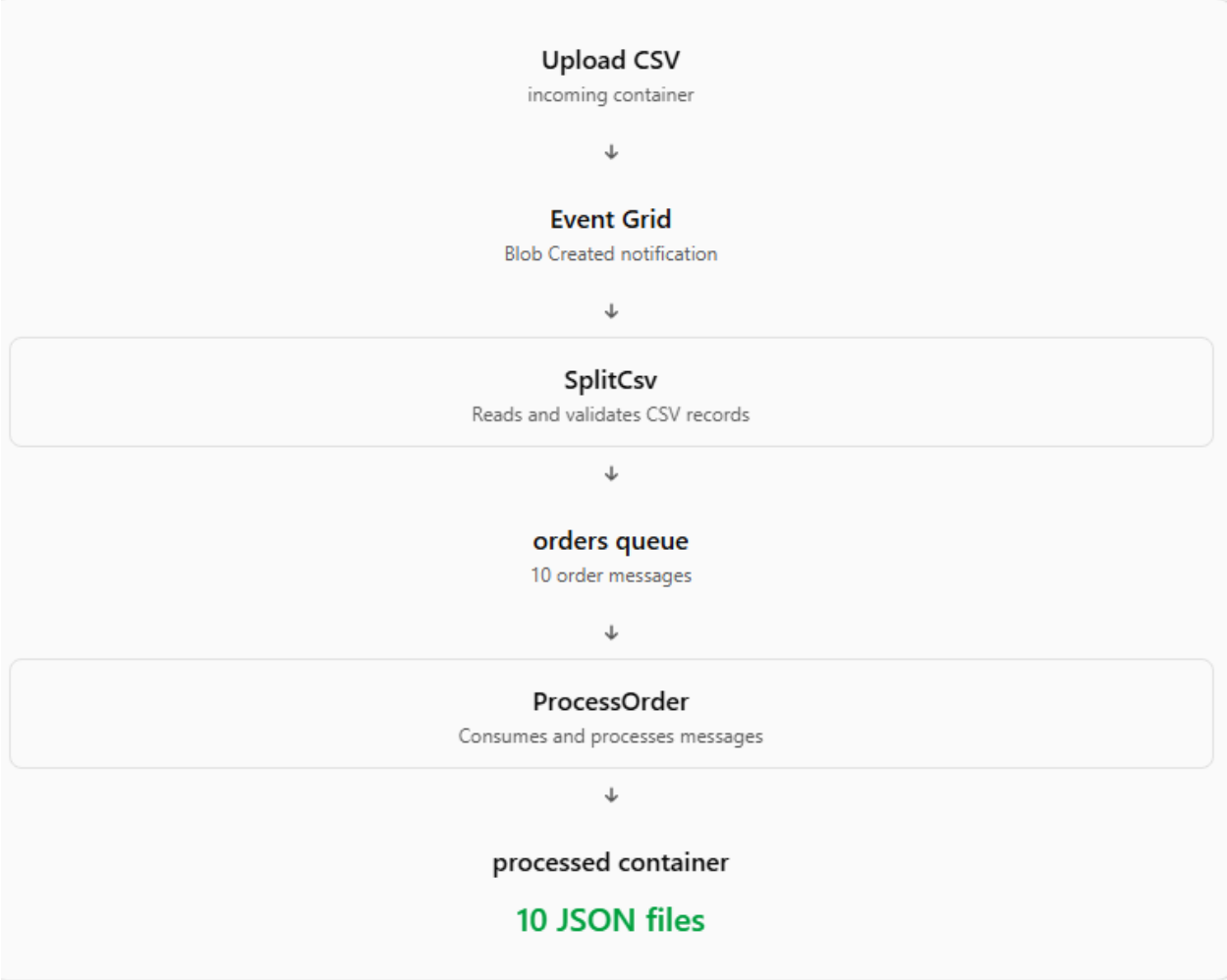
7. Open Function App → `ProcessOrder` → Monitor.
8. Verify that queue messages are being processed.



9. Open Storage Account → Containers → `processed`.
10. Refresh and count the generated JSON file



Expected processing flow:



10 JSON files

12. Validate monitoring

Open the Function App's Application Insights resource → Logs.

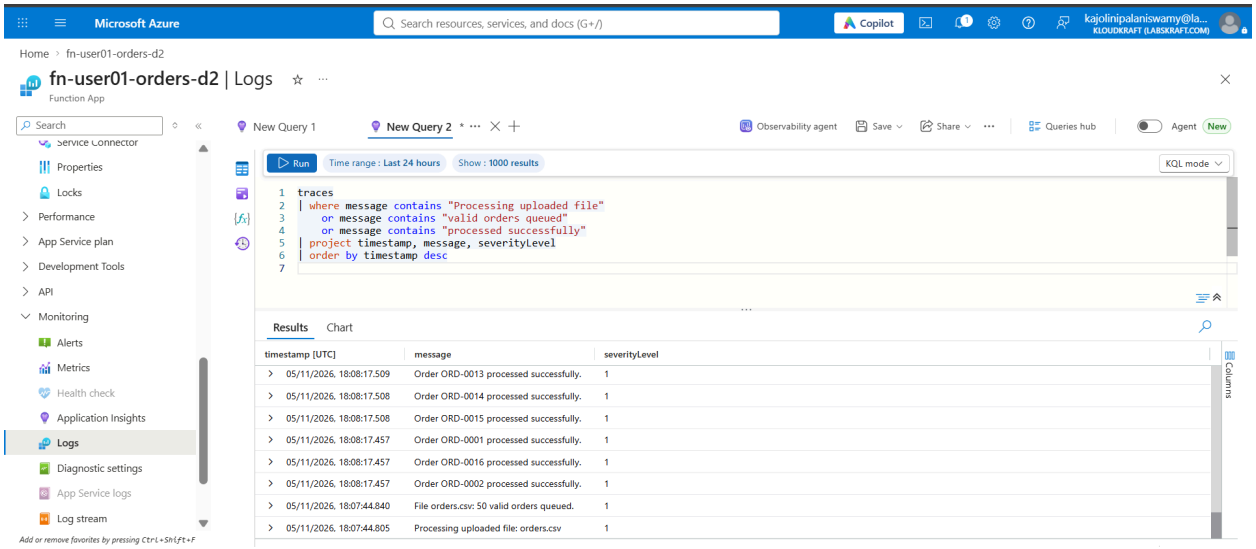
Run:

None

```
traces
| where message contains "Processing uploaded file"
    or message contains "valid orders queued"
    or message contains "processed successfully"
| project timestamp, message, severityLevel
| order by timestamp desc
```

Expected:

- CSV processing log from `SplitCsv`.
- Valid order count.
- Individual order-processing logs from `ProcessOrder`.



13. Troubleshooting

Issue	What to check
Event subscription creation fails	Endpoint URL, system key and Event Grid permissions
Subscription validation fails	Function App running and webhook URL correct
Blob upload does not trigger	Event Grid subscription state and subject filters
`SplitCsv` does not execute	`Source = BlobTriggerSource.EventGrid` deployed correctly
Queue remains empty	Blob trigger logs, Storage connection and CSV headers
JSON files not generated	Queue connection, `ProcessOrder` logs and output container

Storage authentication error	Function App's Blob and Queue permissions

Important: Event Grid delivery is asynchronous. Allow time for delivery and inspect the subscription's delivery metrics if events are not reaching the endpoint.

14. Acceptance criteria

Validation	Expected result
Function App	Day 2 Function App running
Blob trigger	Event Grid source configured
Deployment	Successful
Event subscription	Provisioning state Succeeded
Subject filter	`incoming` and `.csv`
Blob upload	New CSV uploaded

SplitCsv	Successfully invoked
Queue processing	10 valid messages processed
ProcessOrder	Successfully invoked
Output	Exactly 10 JSON files
Monitoring	Execution logs visible

15. End-of-day cleanup

Perform after completing all Day 2 labs

1. Open the shared Resource Group.
2. Identify your Day 2 resources.
3. Delete your Event Grid subscription if it remains separately listed.
4. Delete your Function App.
5. Delete your Day 2 Storage Account.
6. Delete dedicated Application Insights and Log Analytics resources, if created exclusively for your lab.
7. Verify deletion.

Do not delete the shared Resource Group or other participants' resources.

Preserve your local C# project, CSV asset and execution evidence.